```
Start coding or generate with AI.
```

Task

Analyze the 'global_sports_footwear_sales_2018_2026.csv' dataset by performing exploratory data analysis (EDA), including visualizing sales distribution, feature correlations, categorical feature impact, and time series trends. Then, preprocess the data by handling missing values, encoding categorical features, and extracting relevant features. Train and hyperparameter tune a Decision Tree model to predict sales, evaluate its performance, and analyze feature importances. Finally, summarize all findings, present the model's performance, and discuss insights and potential next steps.

Load Data

Subtask:

Load the 'global_sports_footwear_sales_2018_2026.csv' dataset into a pandas DataFrame and display its head, info, and basic descriptive statistics.

Reasoning: To load the dataset and display its head, info, and descriptive statistics, I need to write a Python code block that imports pandas, reads the CSV file into a DataFrame, and then calls the .head(), .info(), and .describe() methods.

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/global_sports_footwear_sales_2018_2026.csv')

# Display the first 5 rows of the DataFrame
print('DataFrame Head:')
print(df.head())
print('\n')

# Print a concise summary of the DataFrame
print('DataFrame Info:')
df.info()
print('\n')

# Generate descriptive statistics of the numerical columns
print('DataFrame Descriptive Statistics:')
print(df.describe())
```

```
   base_price_usd  discount_percent  final_price_usd  units_sold  revenue_usd  \
0             162                15           137.70           1        137.7
1              80                 5            76.00           3        228.0
2             176                15           149.60           4        598.4
3              61                15            51.85           2        103.7
4              80                 0            80.00           4        320.0

  payment_method sales_channel  country customer_income_level  customer_rating
0           Card  Retail Store  Germany                   Low              4.6
1           Card        Online      USA                   Low              3.9
2           Cash  Retail Store    India                Medium              3.0
3           Card  Retail Store    India                  High              3.4
4           Cash        Online      USA                Medium              3.0


DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   order_id              30000 non-null  object
 9   discount_percent      30000 non-null  int64
```

```
 10  final_price_usd        30000 non-null  float64
 11  units_sold             30000 non-null  int64
 12  revenue_usd            30000 non-null  float64
 13  payment_method         30000 non-null  object
 14  sales_channel          30000 non-null  object
 15  country                30000 non-null  object
 16  customer_income_level  30000 non-null  object
 17  customer_rating        30000 non-null  float64
dtypes: float64(3), int64(4), object(11)
memory usage: 4.1+ MB


DataFrame Descriptive Statistics:
                size  base_price_usd  discount_percent  final_price_usd  \
count  30000.000000    30000.000000      30000.000000     30000.000000
mean       8.500867      139.634633         13.332167       121.029035
std        1.710896       46.062549          9.864198        42.511586
min        6.000000       60.000000          0.000000        42.000000
25%        7.000000      100.000000          5.000000        85.000000
50%        9.000000      140.000000         10.000000       119.200000
75%       10.000000      180.000000         20.000000       153.600000
max       11.000000      219.000000         30.000000       219.000000

         units_sold   revenue_usd  customer_rating
count  30000.000000  30000.000000     30000.000000
mean       2.500200    302.714948         4.001543
std        1.121149    179.149272         0.577546
```

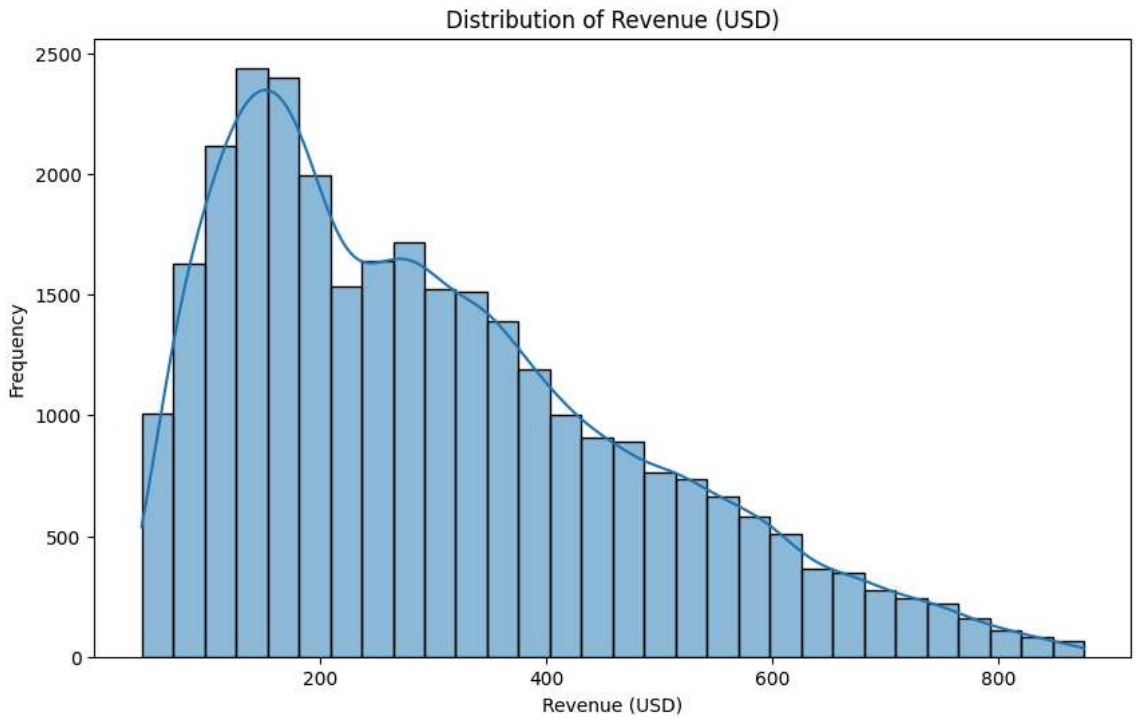
EDA - Sales Distribution

Subtask:

Visualize the distribution of the target variable (assuming 'Sales' or similar) using a histogram or a box plot to understand its spread and identify potential outliers.

Reasoning: To visualize the distribution of 'revenue_usd', I will create a histogram using `matplotlib.pyplot` and `seaborn` as instructed. This will help in understanding its spread and identifying potential outliers.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a histogram of the 'revenue_usd' column
plt.figure(figsize=(10, 6))
sns.histplot(df['revenue_usd'], bins=30, kde=True)
plt.title('Distribution of Revenue (USD)')
plt.xlabel('Revenue (USD)')
plt.ylabel('Frequency')
plt.show()
```



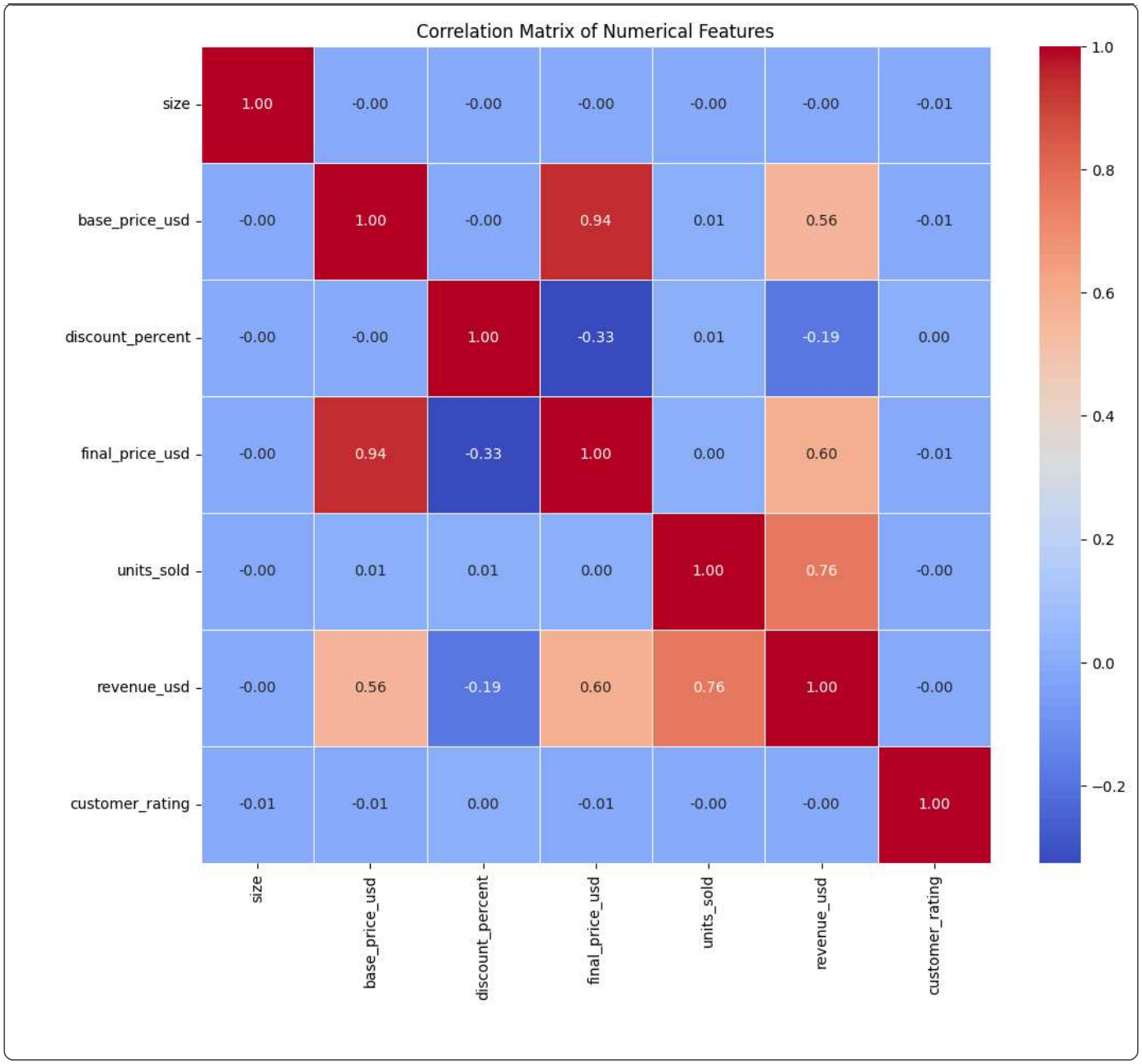
EDA - Feature Correlation

Subtask:

Generate a correlation heatmap to analyze the relationships between numerical features in the dataset, providing insights into potential multicollinearity.

Reasoning: To generate a correlation heatmap, I need to first select all numerical columns from the DataFrame, calculate their correlation matrix, and then visualize this matrix using `seaborn.heatmap` with appropriate settings for readability.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Select numerical columns
numerical_cols = df.select_dtypes(include=['number'])

# Calculate the correlation matrix
correlation_matrix = numerical_cols.corr()

# Create a heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



EDA - Categorical Feature Analysis

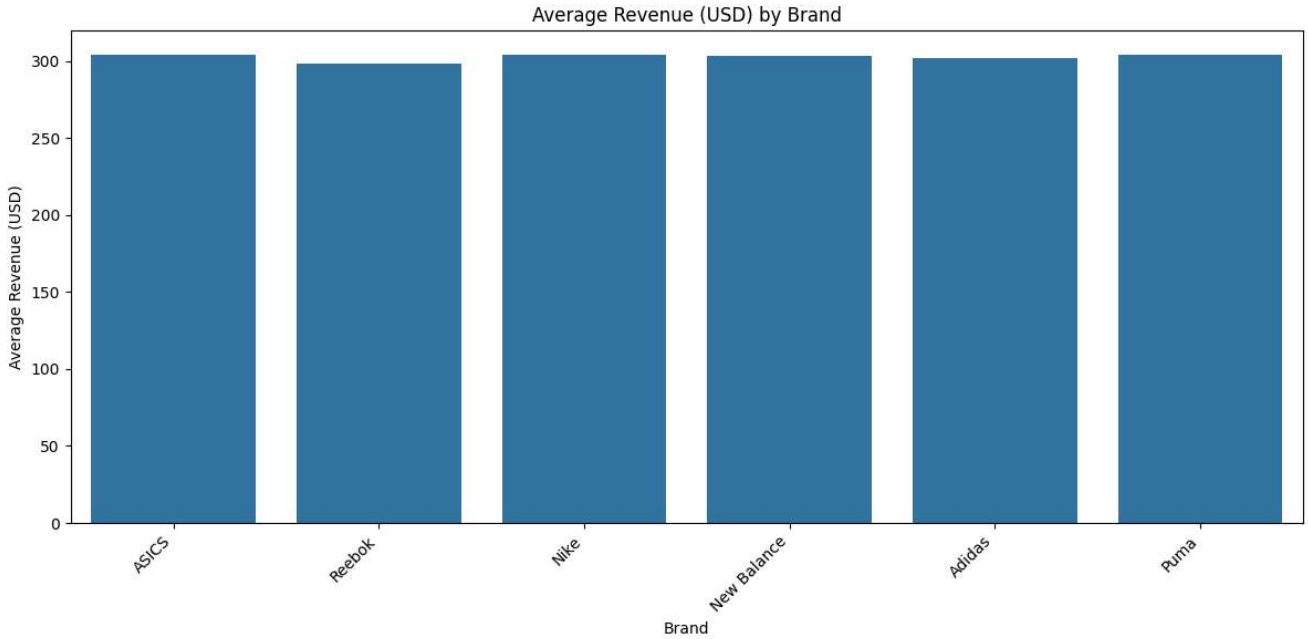
Subtask:

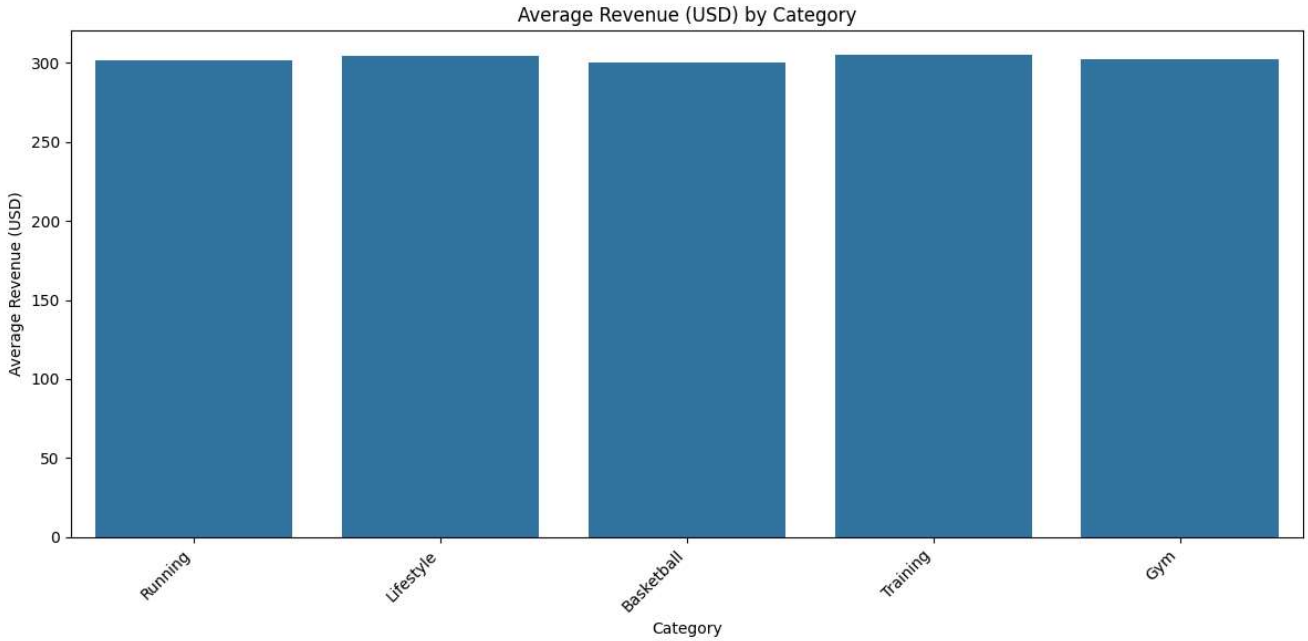
Explore the distribution of key categorical features, such as 'brand', 'category', 'gender', 'customer_income_level', 'sales_channel', and 'country', to understand their impact on 'revenue_usd'.

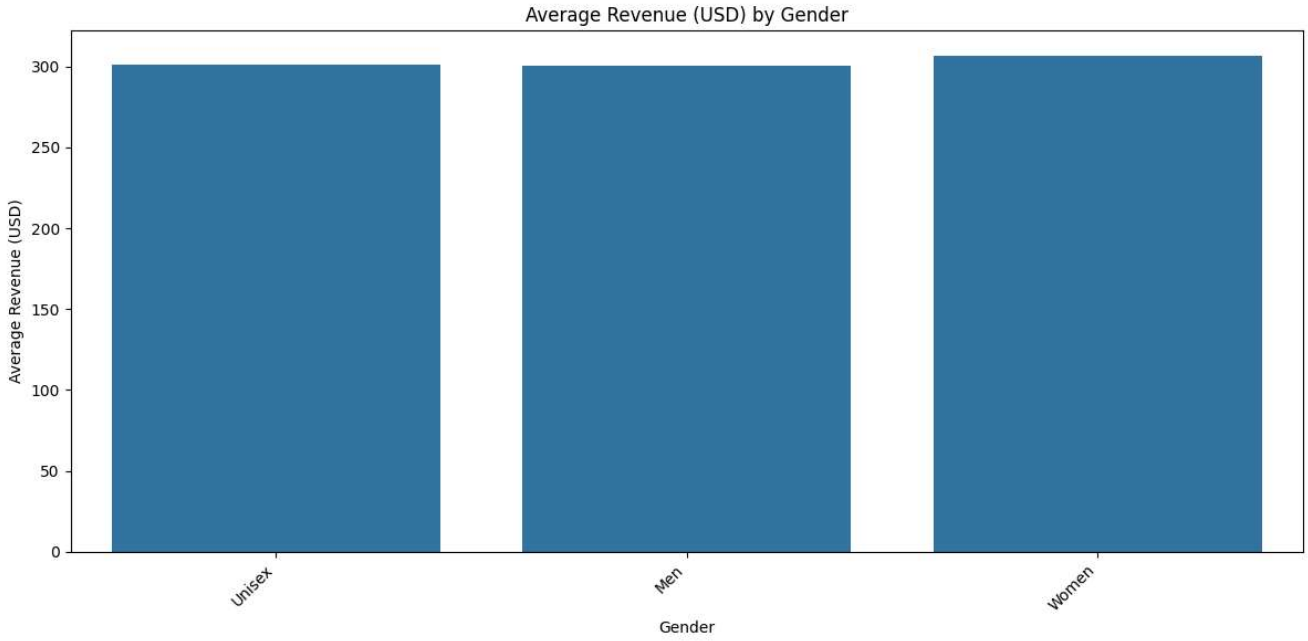
Reasoning: To analyze the impact of categorical features on 'revenue_usd', I will iterate through the specified categorical columns and create a bar plot for each, displaying the average 'revenue_usd' per category. This will involve using `seaborn.barplot` and `matplotlib.pyplot` for visualization and setting appropriate titles and labels.

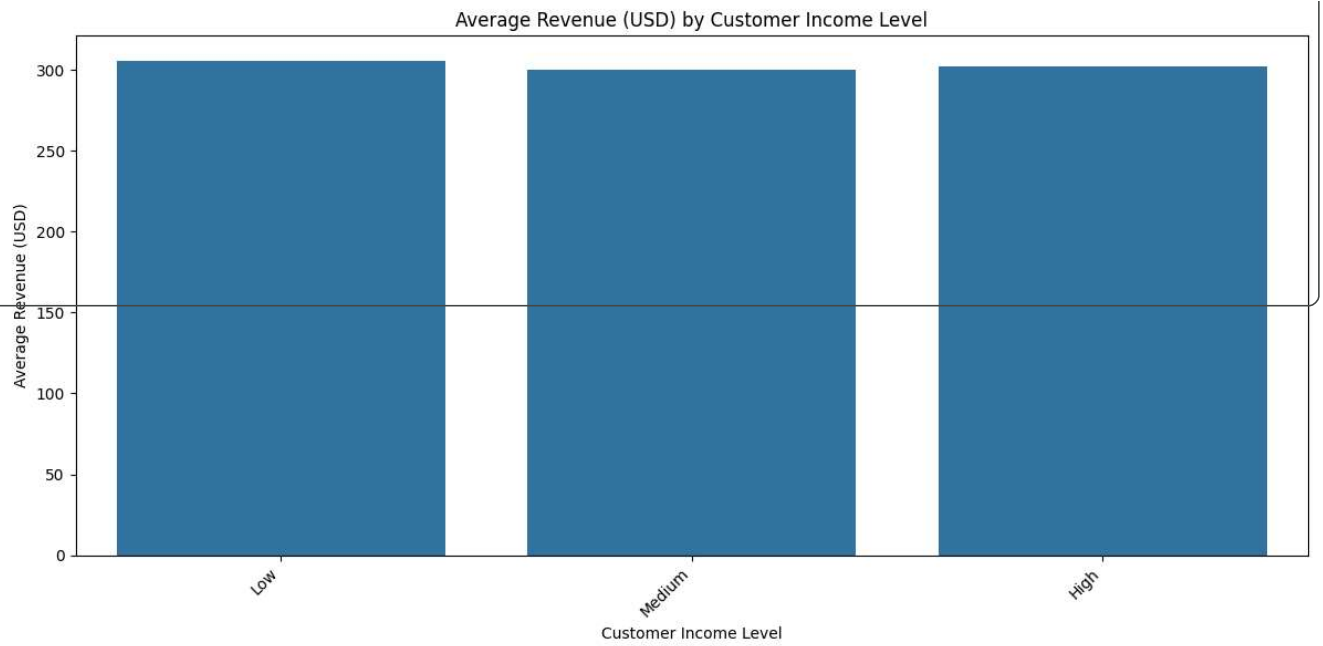
```
import matplotlib.pyplot as plt
import seaborn as sns

# List of categorical features to analyze
categorical_features = [
    'brand',
    'category',
    'gender',
    'customer_income_level',
    'sales_channel',
```

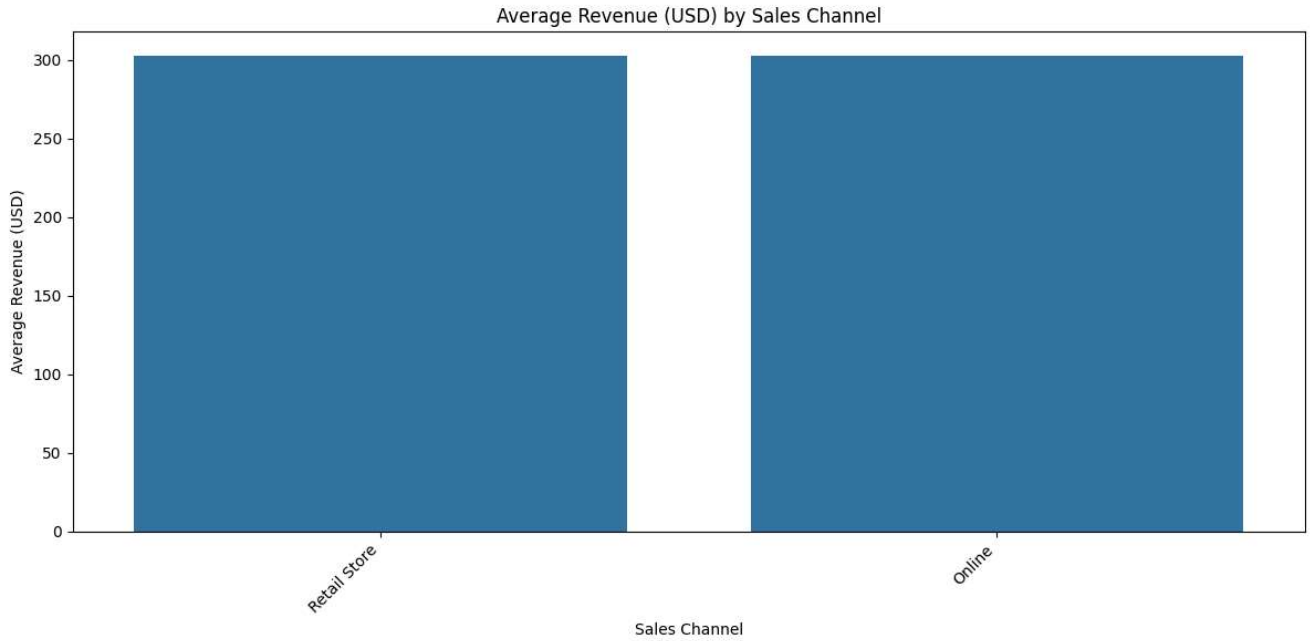
```
    'country'
]

# Create bar plots for each categorical feature against 'revenue_usd'
for feature in categorical_features:
    plt.figure(figsize=(12, 6))
    sns.barplot(x=feature, y='revenue_usd', data=df, errorbar=None)
    plt.title(f'Average Revenue (USD) by {feature.replace("_", " ").title()}')
    plt.xlabel(feature.replace("_", " ").title())
    plt.ylabel('Average Revenue (USD)')
    plt.xticks(rotation=45, ha='right') # Rotate labels for better readability
    plt.tight_layout()
    plt.show()
```

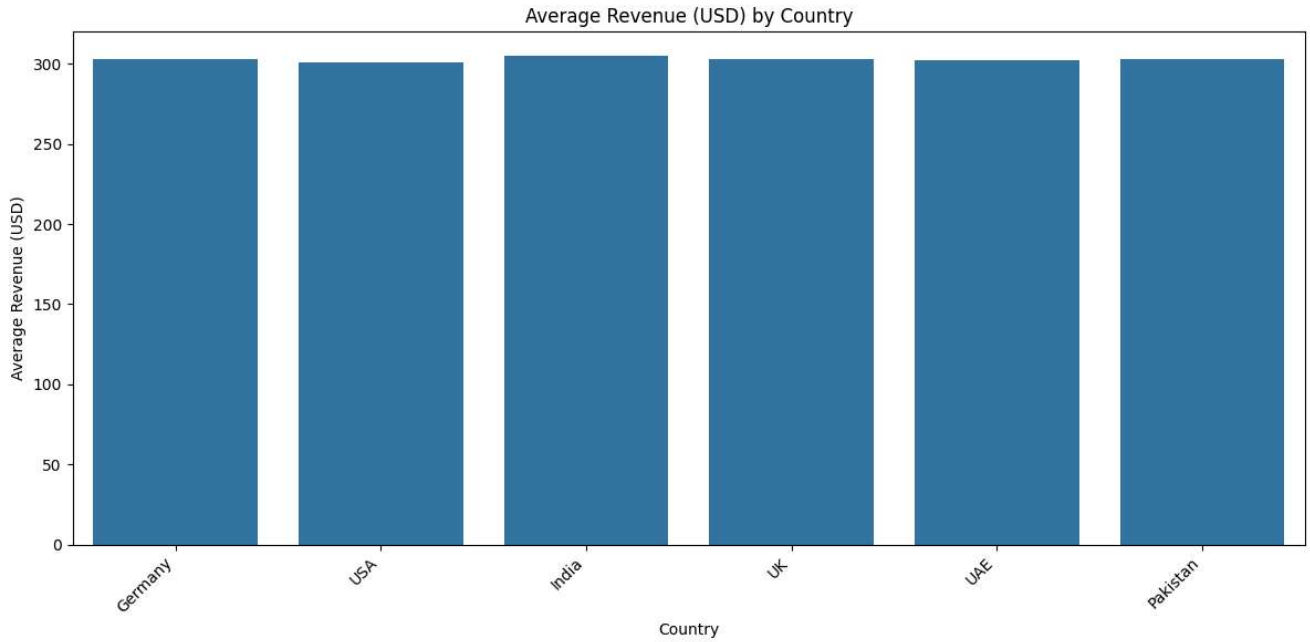

Average Revenue (USD) by Brand
Average Revenue (USD)
0
50
100
150
200
250
300
ASICS
Reebok
Nike
New Balance
Adidas
Puma
Brand


Average Revenue (USD) by Category
Average Revenue (USD)
0
50
100
150
200
250
300
Running
Lifestyle
Basketball
Training
Gym
Category


Average Revenue (USD) by Gender
Average Revenue (USD)
0
50
100
150
200
250
300
Unisex
Men
Women
Gender


Average Revenue (USD) by Customer Income Level
Average Revenue (USD)
0
50
100
150
200
250
300
Low
Medium
High
Customer Income Level


Average Revenue (USD) by Sales Channel
Average Revenue (USD)
0
50
100
150
200
250
300
Retail Store
Online
Sales Channel


Average Revenue (USD) by Country
Average Revenue (USD)
0
50
100
150
200
250
300
Germany
USA
India
UK
UAE
Pakistan
Country

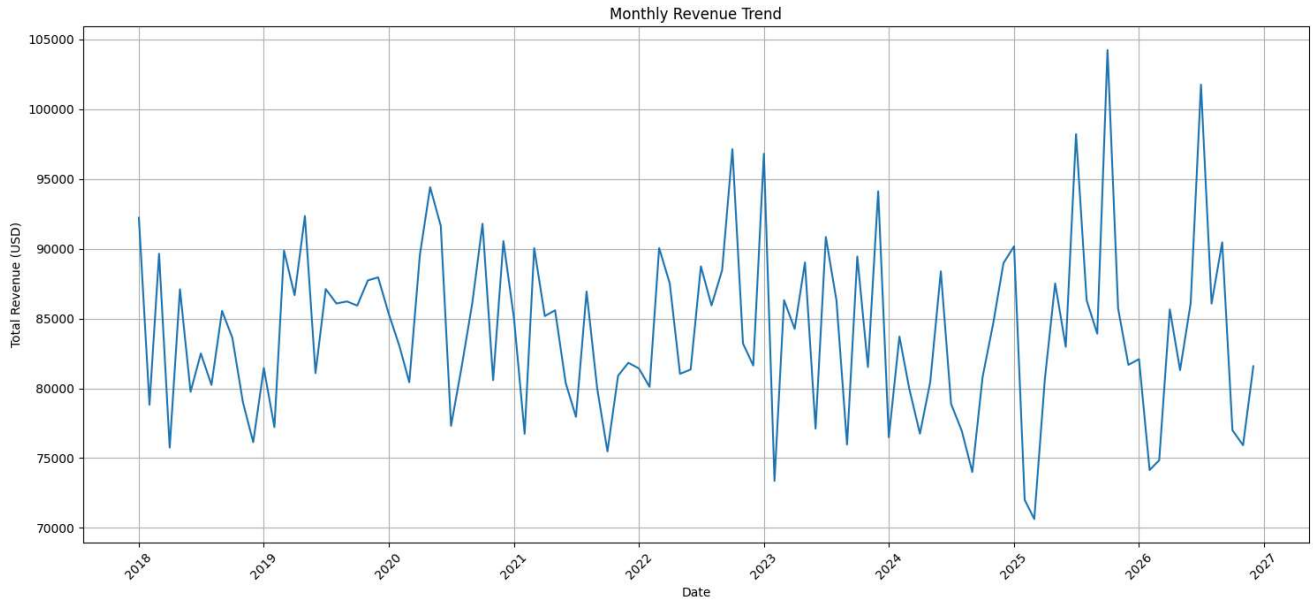
EDA - Time Series Trend

Subtask:

If a date column is available, visualize the sales trend over time to identify seasonality or growth patterns. Otherwise, analyze another relevant feature's trend.

Reasoning: To visualize the sales trend over time, I will first convert the 'order_date' column to datetime objects. Then, I will extract the month and year to group the data, calculate the sum of 'revenue_usd' for each period, and finally create a line plot to display the monthly revenue trend.

```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Convert 'order_date' to datetime objects
df['order_date'] = pd.to_datetime(df['order_date'])

# 2. Extract year and month to create 'order_month_year'
df['order_month_year'] = df['order_date'].dt.to_period('M')

# 3. Group by 'order_month_year' and calculate the sum of 'revenue_usd'
monthly_revenue = df.groupby('order_month_year')['revenue_usd'].sum().reset_index()

# Convert 'order_month_year' back to datetime for plotting purposes
monthly_revenue['order_month_year'] = monthly_revenue['order_month_year'].dt.to_timestamp()

# 4. Create a line plot to visualize the 'revenue_usd' trend over time
plt.figure(figsize=(15, 7))
sns.lineplot(x='order_month_year', y='revenue_usd', data=monthly_revenue)

# 5. Set plot title and labels
plt.title('Monthly Revenue Trend')
plt.xlabel('Date')
plt.ylabel('Total Revenue (USD)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Feature Engineering and Preprocessing

Subtask:

Handle any missing values, encode categorical features using appropriate techniques like One-Hot Encoding, and if applicable, extract new features from date columns (e.g., year, month).

Reasoning: First, I will check for any missing values in the DataFrame to ensure data quality and confirm previous observations. Then, I will extract year, month, and day of week from the 'order_date' column to create new numerical features, and finally, I will identify and apply one-hot encoding to all suitable categorical columns, while dropping the original categorical and date columns.

```
print("Missing values before preprocessing:")
print(df.isnull().sum())

# 1. Handle any missing values (based on df.info(), there are none, but we'll confirm)
# If there were missing values, a strategy like df.fillna() or df.dropna() would be used here.
# For now, we confirm there are no missing values.

# 4. Extract new features from 'order_date'
df['order_year'] = df['order_date'].dt.year
df['order_month'] = df['order_date'].dt.month
df['order_day_of_week'] = df['order_date'].dt.dayofweek # Monday=0, Sunday=6

# 2. Identify categorical columns for encoding (excluding 'order_id')
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
# Exclude 'order_id' as it's a unique identifier, not a feature for encoding
if 'order_id' in categorical_cols:
    categorical_cols.remove('order_id')

# 3. Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# 5. Drop original 'order_date' and 'order_month_year' columns
df_processed = df_encoded.drop(columns=['order_date', 'order_month_year'], errors='ignore')

print("\nDataFrame head after preprocessing:")
print(df_processed.head())

print("\nDataFrame info after preprocessing:")
df_processed.info()
```

```
category                 0
gender                   0
size                     0
color                    0
base_price_usd           0
discount_percent         0
final_price_usd          0
units_sold               0
revenue_usd              0
payment_method           0
sales_channel            0
country                  0
customer_income_level    0
customer_rating          0
order_month_year         0
dtype: int64

DataFrame head after preprocessing:
    order_id  size  base_price_usd  discount_percent  final_price_usd  \
0  ORD100000     8             162                15           137.70
1  ORD100001     8              80                 5            76.00
2  ORD100002     8             176                15           149.60
3  ORD100003     7              61                15            51.85
4  ORD100004    11              80                 0            80.00

   units_sold  revenue_usd  customer_rating  order_year  order_month  ...  \
0           1        137.7              4.6        2021            1  ...
1           3        228.0              3.9        2026           10  ...
2           4        598.4              3.0        2023           11  ...
3           2        103.7              3.4        2026            8  ...
4           4        320.0              3.0        2019           11  ...

   payment_method_Cash  payment_method_Wallet  sales_channel_Retail Store  \
0                False                  False                        True
1                False                  False                       False
```

```
1          False             False        False       False         True
2           True             False        False       False        False
3           True             False        False       False        False
4          False             False        False       False         True

   customer_income_level_Low  customer_income_level_Medium
0                       True                         False
1                       True                         False
2                      False                          True
3                      False                         False
4                      False                          True

[5 rows x 935 columns]

DataFrame info after preprocessing:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
```

Split Data

Subtask:

Split the preprocessed dataset into training and testing sets to prepare for model training and unbiased evaluation.

Reasoning: To split the preprocessed dataset into training and testing sets, I will first define the target variable and feature matrix, then import `train_test_split` and use it to perform the split with a specified `test_size` and `random_state`.

```
from sklearn.model_selection import train_test_split

# Define the target variable y
y = df_processed['revenue_usd']

# Define the feature matrix X by dropping 'revenue_usd' and 'order_id'
X = df_processed.drop(columns=['revenue_usd', 'order_id'])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

```
Shape of X_train: (24000, 933)
Shape of X_test: (6000, 933)
Shape of y_train: (24000,)
Shape of y_test: (6000,)
```

Train Initial Decision Tree Model

Subtask:

Train a baseline Decision Tree Regressor or Classifier model with default parameters on the training data.

Reasoning: To train a baseline Decision Tree Regressor model, I will import `DecisionTreeRegressor` from `sklearn.tree`, instantiate it with `random_state=42`, and then fit it to the `X_train` and `y_train` datasets.

```
from sklearn.tree import DecisionTreeRegressor

# Instantiate a DecisionTreeRegressor object with random_state=42
dt_model = DecisionTreeRegressor(random_state=42)

# Train the model using the .fit() method
dt_model.fit(X_train, y_train)

print("Decision Tree Regressor model trained successfully.")
```

```
Decision Tree Regressor model trained successfully.
```

Evaluate Initial Model

Subtask:

Evaluate the performance of the initial decision tree model using relevant metrics such as R-squared, Mean Absolute Error (MAE), or Mean Squared Error (MSE) for regression, or accuracy, precision, recall, and F1-score for classification.

Reasoning: To evaluate the initial Decision Tree model, I will use the trained model to make predictions on the test set (`X_test`), then calculate and print the Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared score using `sklearn.metrics`.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Make predictions on the X_test dataset
y_pred = dt_model.predict(X_test)

# 3. Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)

# 4. Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# 5. Calculate R-squared score
r2 = r2_score(y_test, y_pred)

# 6. Print the calculated metrics
print(f"Model Performance on Test Set:")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared (R2): {r2:.2f}")
```

```
Model Performance on Test Set:
Mean Absolute Error (MAE): 0.00
Mean Squared Error (MSE): 0.00
R-squared (R2): 1.00
```

Hyperparameter Tuning

Subtask:

Perform hyperparameter tuning on the Decision Tree model using techniques like GridSearchCV or RandomizedSearchCV to find the optimal combination of parameters for improved performance.

Reasoning: To perform hyperparameter tuning on the Decision Tree model, I will use `GridSearchCV` as specified in the instructions. This involves defining a parameter grid, instantiating `GridSearchCV`, fitting it to the training data, and then extracting the best parameters and score.

Reasoning: The previous code block contained a `SyntaxError` due to an unmatched parenthesis in the `print` statement for the best score. I will correct this syntax error to allow the code to execute properly.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor

# Define the parameter grid for DecisionTreeRegressor
param_grid = {
    'max_depth': [5, 7, 10, 12, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Instantiate DecisionTreeRegressor with random_state
dt = DecisionTreeRegressor(random_state=42)

# Instantiate GridSearchCV
grid_search = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    cv=5, # 5-fold cross-validation
    scoring='neg_mean_squared_error', # Use negative MSE for optimization
```